

Recreating VoiceFilter-Lite for Whisper-Based Target-Speaker ASR: Current Training Method

Gal Abuhatzira
211645361
abuhatzira@mail.tau.ac.il

Eden Aharon
318382017
edenal@mail.tau.ac.il

Michael Lipkin
318815800
lipkin1@mail.tau.ac.il

David Zagorelski
322292152
zagorelski@mail.tau.ac.il

Abstract

We present an accessible, Windows-ready reproduction of Google’s VoiceFilter-Lite: a lightweight, speaker-conditioned frontend designed to isolate a target speaker’s voice prior to automatic speech recognition (ASR). Our reproduction maintains the core architecture of the original work—incorporating an enrollment embedding, a lightweight LSTM enhancement model, ASR-feature-domain processing, and an asymmetric feature loss. However, we adapt the framework for open research by replacing proprietary production components with modern, open-source equivalents, notably utilizing frozen Whisper ASR features, SpeechBrain ECAPA speaker embeddings, and offline PyTorch training on Libri2Mix two-speaker mixtures. Through our evaluations, we identify that training sequence crop length is a critical hyperparameter for successful model convergence. Using an optimized 1200-frame configuration, our model yields strong early results on a same-seed 500-example development subset, reducing noisy-mixture WER from 70.17 % to 45.29 % by epoch 5, confirming the viability of this architecture using accessible research tools.

1 Introduction

Automatic speech recognition (ASR) in overlapping speech remains a practical version of the cocktail-party problem [2]. Blind source-separation methods such as deep clustering, deep attractor networks, and permutation-invariant training made major progress on separating multi-speaker mixtures [7, 1, 13]. Those systems, however, usually separate all speakers and then require an additional decision about which output stream should be recognized. Voice filtering, or target-speaker extraction, changes the problem: the desired speaker is known in advance through enrollment, so the separator can condition on a speaker representation such as an i-vector or d-vector [4, 9]. Prior speaker-conditioned systems, including speaker-aware beamforming, DENet, SpeakerBeam, and the original VoiceFilter, use this tar-

get identity to avoid the source-count, permutation, and output-selection issues of blind separation [14, 10, 5, 12].

Google’s VoiceFilter-Lite paper [11] addresses a stricter production setting: target-speaker separation for streaming, on-device ASR. The model must improve recognition when another speaker overlaps the target user, but it must also avoid harming recognition when the input is already clean or contains only non-speech noise. It must be causal, small, fast, and cheap enough for CPU, memory, battery, and latency budgets. To solve this, VoiceFilter-Lite is placed directly in front of the ASR model as a frame-by-frame feature processor rather than an audio-reconstruction system. Given ASR acoustic features and a stored target-speaker d-vector, it predicts enhanced features using a lightweight network with streaming-compatible LSTM layers. The paper also introduces two important safety mechanisms: an asymmetric feature loss that penalizes over-suppression more strongly than under-suppression, and an adaptive suppression strength that can reduce filtering when the model predicts that filtering is unnecessary. The production version is further optimized through 8-bit TensorFlow Lite quantization.

This project recreates the central VoiceFilter-Lite idea in an accessible Whisper-based research setting. Instead of Google’s streaming RNN-T recognizer, we use frozen Whisper small.en features [8]; instead of Google’s d-vector speaker encoder, we use SpeechBrain ECAPA speaker embeddings [6]; and instead of Google’s LibriSpeech plus vendor-collected query data with speech, non-speech, and reverberant noise simulation, we train on generated Libri2Mix two-speaker mixtures [3]. We also train and evaluate an offline PyTorch residual frontend over Whisper’s 80-bin log-mel features rather than a quantized TensorFlow Lite streaming model. The goal of this report is therefore not to claim an identical production reproduction, but to test whether the VoiceFilter-Lite approach remains viable when rebuilt from open-source components, whether the resulting system can be recreated by other researchers, and how far its early WER improvements are from the behavior reported in Google’s original system.

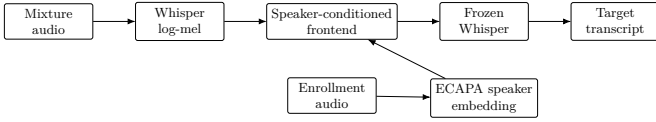


Figure 1: Reproduced VoiceFilter-Lite-style Whisper pipeline. Only the speaker-conditioned frontend is trained; Whisper remains frozen.

2 Method

2.1 High-Level Architecture

Figure 1 shows the reproduced VoiceFilter-Lite-style pipeline. A two-speaker mixture is converted to Whisper-compatible log-mel features. A speaker encoder extracts an enrollment embedding from a clean reference utterance of the target speaker. The trainable frontend receives the mixture features together with this target-speaker embedding and predicts enhanced ASR features before recognition. Whisper is kept frozen, so the only learned component is the small speaker-conditioned frontend.

The original VoiceFilter-Lite paper describes three possible ASR integration points: FFT magnitude features, filterbank energies, or stacked filterbank energies [11]. Our reproduction uses the filterbank-feature option. This choice follows from the use of Whisper as the frozen ASR backend: public Whisper models consume 80-bin log-mel features, while the exact stacked-filterbank interface used in Google’s on-device ASR setup is not available as a comparable public pretrained ASR target. Operating at the log-mel level therefore keeps the experiment reproducible while staying close to the VoiceFilter-Lite feature-enhancement design.

2.2 Open Components and Data

The method is built from open research components. The ASR model is frozen Whisper small.en [8]. Target-speaker identity is represented by 192-dimensional Speech-Brain ECAPA embeddings [6], extracted on CPU and cached. Training data are generated Libri2Mix clean two-speaker mixtures [3], built from LibriSpeech utterances and Libri2Mix metadata. Each mixture provides a noisy `mix_clean` input, a clean target source, a clean interferer source, and an enrollment utterance for the target speaker. The implementation is an offline PyTorch training setup on Windows; it does not implement Google’s quantized TensorFlow Lite streaming deployment.

2.3 Frontend Model

For each time frame, the frontend concatenates normalized 80-bin Whisper log-mel features with the target-speaker embedding. The active model uses hidden size 384, three unidirectional LSTM layers, dropout 0.05, and a residual output head. In our reproduction, we implement the enhancement frontend as a bounded residual pre-

dictor rather than as a purely multiplicative mask. This choice is motivated by the use of a strong frozen ASR backend: Whisper already extracts robust speech features, so the frontend should make targeted corrections for speaker interference without freely rewriting the representation or over-suppressing target speech cues. The model predicts

$$\hat{x}_{t,f} = x_{t,f} + \Delta_{t,f}, \quad \Delta_{t,f} = r_{\max} \tanh(z_{t,f}), \quad (1)$$

where x is the noisy Whisper log-mel feature, $\hat{x}$ is the enhanced feature, and $z_{t,f}$ is the unbounded residual logit predicted by the frontend for time frame t and feature bin f . The tanh nonlinearity bounds the correction because it stays between -1 and 1 , so the model can change each feature bin by at most $r_{\max}$. The current full-training recipe uses $r_{\max} = 0.35$, meaning each Whisper log-mel bin can be nudged up or down but not rewritten arbitrarily.

2.4 Training Objective and Hyperparameters

The main training metric is a feature-domain reconstruction loss between enhanced Whisper features and clean target-speaker Whisper features. As in VoiceFilter-Lite, the core term is asymmetric:

$$g_{\alpha}(d) = \begin{cases} \alpha d, & d > 0, \\ d, & d \leq 0, \end{cases} \quad (2)$$

and

$$L_{\text{asym}} = \sum_{t,f} g_{\alpha}(S_{\text{clean}}(t,f) - S_{\text{enh}}(t,f))^2, \quad (3)$$

so positive clean-minus-enhanced errors are upweighted to penalize over-suppression. We keep this asymmetric reconstruction term on Whisper log-mel features, but the total objective also includes identity regularization, temporal smoothness on the predicted control signal, and binary overlap supervision:

$$L_{\text{ours}} = L_{\text{asym}} + \lambda_{\text{id}} L_{\text{id}} + \lambda_{\text{sm}} L_{\text{smooth}} + \lambda_{\text{aux}} L_{\text{overlap}}. \quad (4)$$

The identity term discourages unnecessary movement when the input is already close to the target, the smoothness term discourages frame-to-frame instability in the predicted control signal, and the auxiliary overlap term supervises whether speech overlap is present. The active 1200-frame training recipe uses batch size 1 with gradient accumulation of 8, learning rate $7 \cdot 10^{-4}$, minimum learning rate 10^{-5} , weight decay 10^{-5} , gradient clipping at 5.0, identity weight 0.01, smoothness weight 0.005, auxiliary overlap weight 0.05, and early stopping patience 6.

2.5 ASR Evaluation

The training loss is only a proxy; final usefulness is measured through ASR. For evaluation, clean source speech, unenhanced mixture speech, enhanced speech, and wrong-speaker enrollment are decoded with the same frozen

Whisper model. The main recognition metric is word error rate,

$$\text{WER} = \frac{S + D + I}{N}, \quad (5)$$

where S , D , and I are word substitutions, deletions, and insertions, and N is the number of reference words. WER is reported for the clean target source, noisy mixture baseline, enhanced output, and wrong-d-vector condition.

To measure improvement over the noisy input, we report relative WER reduction,

$$\text{rel WER red.} = \frac{\text{WER}_{\text{noisy}} - \text{WER}_{\text{enh}}}{\text{WER}_{\text{noisy}}}. \quad (6)$$

This metric answers how much of the noisy-mixture ASR error is removed by the frontend. We also report oracle gap closed,

$$\text{oracle gap} = \frac{\text{WER}_{\text{noisy}} - \text{WER}_{\text{enh}}}{\text{WER}_{\text{noisy}} - \text{WER}_{\text{clean}}}, \quad (7)$$

which compares the achieved improvement to the maximum possible improvement suggested by the clean target-source reference. The wrong-speaker condition is included to test whether the frontend is actually using the enrollment embedding rather than acting as a generic denoiser.

2.6 Implementation Challenges

Several practical challenges shaped the final method. First, the architecture had to be redesigned around open components rather than Google’s production stack. This required choosing compatible public models for ASR and speaker embedding, then tuning the frontend so that ECAPA speaker embeddings and Whisper log-mel features could work together as a speaker-conditioned enhancement system. Second, the LibriMix-style dataset is large enough that operating-system-level issues became important: repeated audio loading, speaker-embedding extraction, and feature computation caused substantial I/O overhead unless paths, caches, and evaluation snapshots were organized carefully. Third, training and evaluation optimize different signals. The model is trained with feature-domain losses and regularizers, but it is judged by downstream Whisper WER, so improvements in the abstract loss do not always translate directly into recognition gains. Finally, hyperparameter tuning was unusually sensitive. Many runs were needed to discover that apparently small choices, including crop length, batch size, gradient accumulation, residual range, and regularization strength, could decide whether the model learned useful target-speaker filtering or stayed close to an identity mapping.

3 Results and Discussion

3.1 Training Loss

Table 1 shows the first five epochs of the 1200-frame Libri2Mix train-100 run. The training loss decreases

Table 1: Training and validation feature losses from train.log.

Epoch	Train	Val	Feature
1	14.3066	16.8390	16.8002
2	12.5728	19.9398	19.8933
3	11.5181	23.9936	23.9404
4	10.8378	23.2116	23.1588
5	10.3213	20.0169	19.9678

Table 2: Same-seed dev-subset ASR results. Clean WER is 3.67% and noisy-mixture WER is 70.17% for every epoch.

Epoch	Enhanced	Wrong emb.	Rel. red.	Oracle gap
1	57.42 %	72.83 %	18.18 %	19.18 %
2	52.08 %	72.70 %	25.78 %	27.21 %
3	50.11 %	72.38 %	28.59 %	30.17 %
4	48.64 %	71.56 %	30.68 %	32.38 %
5	45.29 %	72.24 %	35.46 %	37.41 %

steadily from 14.3066 to 10.3213, showing that the frontend is fitting the training objective. In contrast, validation feature loss increases after epoch 1 before improving again at epoch 5.

3.2 ASR Evaluation

Table 2 reports checkpoint-level WER on the same dev subset for all five epochs. The evaluation uses 500 target-speaker examples, batch size 8, and seed 42. The clean-source WER is 3.67% and the noisy-mixture WER is 70.17% for all rows because the same subset and ASR model are used throughout.

The ASR results tell a different story from the validation feature loss. Although the validation feature metric worsens after epoch 1, downstream WER improves monotonically through epoch 5, from 57.42% enhanced WER to 45.29%. Epoch 5 is the best checkpoint so far: validation loss turns downward, enhanced WER drops by another 3.35% absolute points, and the wrong-embedding condition worsens slightly. This is the central observation of the current run: the feature-domain objective is useful for training, but it is not a sufficient model-selection criterion for a frozen Whisper backend. The final checkpoint choice should therefore be guided by ASR evaluation, not only by validation feature loss.

The wrong-embedding condition remains worse than the noisy baseline for all five epochs. This is important because it suggests that the frontend is not merely acting as a generic denoiser or smoother. With the correct enrollment embedding, the model improves WER substantially; with an incorrect speaker embedding, performance stays close to or worse than the unenhanced mixture. The growing relative WER reduction and oracle gap closed also

show that the model is moving in the desired direction, recovering about 35.46 % of the noisy-mixture WER and closing 37.41 % of the gap toward clean-source recognition by epoch 5.

4 Broader Impact and Risks

Target-speaker enhancement has clear positive uses. It can improve ASR accessibility in meetings, classrooms, assistive devices, and noisy home environments, especially when the user has explicitly enrolled their own voice. An open-source reproduction also has scientific value: VoiceFilter-Lite was originally demonstrated in a production Google setting with components that are not fully available to outside researchers. Rebuilding the idea with public models, public datasets, and reproducible Windows-ready tooling makes the method easier to study, audit, teach, and extend.

The same capability also creates misuse risks. A system that can isolate a target speaker from overlapping speech could be used for eavesdropping, surveillance, or transcription of conversations without the speaker’s consent. This risk is especially important because speaker enrollment can make a model focus on a particular person. For that reason, deployments of this technology should require informed consent from the enrolled speaker, avoid covert collection of enrollment audio, and be evaluated under privacy-preserving data policies. In this work, the experiments are limited to public speech datasets and synthetic two-speaker mixtures, and the intended contribution is research reproducibility rather than covert listening or surveillance.

5 Conclusion

While these results remain preliminary—utilizing a 500-example development subset rather than the full evaluation sets and analyzing early checkpoints from a single run—they strongly support the main methodological claim of this reproduction. With longer 1200-frame crops and less restrictive regularization, the open-source Whisper-based VoiceFilter-Lite frontend can successfully learn speaker-conditioned enhancement that improves downstream ASR. Future work will focus on training the model to full convergence and expanding the evaluation to the complete Libri2Mix test sets to confirm these early findings. Another promising direction is to replace or augment the LSTM frontend with a transformer-based architecture. This may be better suited to the long 1200-frame contexts used in the successful runs, since transformers are designed to model long-range dependencies more directly than recurrent networks. The tradeoff is complexity: standard self-attention has memory cost that scales quadratically with sequence length, making it much less cheap for long contexts. Since VoiceFilter-Lite is intended to remain compact, low-latency, and inexpensive to run,

any transformer variant would need careful design to preserve the lightweight character of the original system.

References

- [1] Zhuo Chen, Yi Luo, and Nima Mesgarani. Deep attractor network for single-microphone speaker separation. In *Proc. ICASSP*, pages 246–250, 2017.
- [2] E. Colin Cherry. Some experiments on the recognition of speech, with one and with two ears. *The Journal of the Acoustical Society of America*, 25(5):975–979, 1953.
- [3] Joris Cosentino, Manuel Pariente, Samuele Cornell, Antoine Deleforge, and Emmanuel Vincent. LibriMix: An open-source dataset for generalizable speech separation, 2020.
- [4] Najim Dehak, Patrick J. Kenny, Reda Dehak, Pierre Dumouchel, and Pierre Ouellet. Front-end factor analysis for speaker verification. *IEEE Transactions on Audio, Speech, and Language Processing*, 19(4):788–798, 2011.
- [5] Marc Delcroix, Katerina Zmolikova, Keisuke Kinoshita, Atsunori Ogawa, and Tomohiro Nakatani. Single channel target speaker extraction and recognition with speaker beam. In *Proc. ICASSP*, pages 5554–5558, 2018.
- [6] Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *Proc. Interspeech*, pages 3830–3834, 2020.
- [7] John R. Hershey, Zhuo Chen, Jonathan Le Roux, and Shinji Watanabe. Deep clustering: Discriminative embeddings for segmentation and separation. In *Proc. ICASSP*, pages 31–35, 2016.
- [8] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022.
- [9] Li Wan, Quan Wang, Alan Papir, and Ignacio Lopez Moreno. Generalized end-to-end loss for speaker verification. In *Proc. ICASSP*, pages 4879–4883, 2018.
- [10] Jiaming Wang, Jie Chen, Dan Su, Lianwu Chen, Meng Yu, Yanmin Qian, and Dong Yu. Deep extractor network for target speaker recovery from single channel speech mixtures, 2018.
- [11] Quan Wang, Ignacio Lopez Moreno, Mert Saglam, Kevin Wilson, Alan Chiao, Renjie Liu, Yanzhang He, Wei Li, Jason Pelecanos, Marily Nika, and Alexander Gruenstein. Voicefilter-lite: Streaming targeted voice

- separation for on-device speech recognition. In Proc. Interspeech 2020, 2020. arXiv:2009.04323.
- [12] Quan Wang, Hannah Muckenhirn, Kevin Wilson, Prashant Sridhar, Zelin Wu, John R. Hershey, Rif A. Saurous, Ron J. Weiss, Ye Jia, and Ignacio Lopez Moreno. Voicefilter: Targeted voice separation by speaker-conditioned spectrogram masking. In Proc. Interspeech, pages 2728–2732, 2019.
- [13] Dong Yu, Morten Kolbaek, Zheng-Hua Tan, and Jesper Jensen. Permutation invariant training of deep models for speaker-independent multi-talker speech separation. In Proc. ICASSP, pages 241–245, 2017.
- [14] Katerina Zmolikova, Marc Delcroix, Keisuke Kinoshita, Takuya Higuchi, Atsunori Ogawa, and Tomohiro Nakatani. Speaker-aware neural network based beamformer for speaker extraction in speech mixtures. In Proc. Interspeech, 2017.